

資料結構

Data Structure

Lab 11

姓名： 紀宥丞

學號： 114310221

Lab11-Ex1.

依照要求完成 Level Sum

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 定義一個常數用來代表陣列中「沒有節點」(空節點)的位置
// 注意：雖然在 C++ 將 NULL 指派給 int 會產生警告，但此處維持原設計作為標記使用
const int EMPTY = NULL;

class TreeNode {
public:
    int value;          // 節點儲存的數值
    TreeNode* left;     // 指向左子節點的指標
    TreeNode* right;    // 指向右子節點的指標

    // 初始化節點數值，並將左右子節點指標預設為 nullptr (空)
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {}
};

class BinaryTree {
public:
    TreeNode* root; // 樹的根節點

    // 初始化時將根節點設為空
    BinaryTree() : root(nullptr) {}

    ~BinaryTree() {
        destroyTree(root);
    }

    // 遞迴函式：搭配解構子使用，利用後序走訪逐一刪除節點
    void destroyTree(TreeNode* node) {
        if (node != nullptr) {
            destroyTree(node->left); // 先清空左子樹
            destroyTree(node->right); // 再清空右子樹
            delete node;             // 最後刪除自己
        }
    }

    TreeNode* buildTree(const vector<int>& arr) {
        // 如果陣列為空，或第一個元素就是空標記，則回傳空樹
        if (arr.empty() || arr[0] == EMPTY) return nullptr;

        // 使用 queue 來輔助建立樹狀結構，裡面存放的是指向節點指標的指標
        // 這樣可以直接修改上一層節點的 left 或 right 指標
        queue<TreeNode*> q;
```

```

root = new TreeNode(arr[0]); // 建立根節點
q.push(&root);              // 將根節點的指標位置推入佇列

size_t i = 1; // 陣列的索引，從 1 開始

// 當佇列不為空，且還沒讀完陣列時繼續迴圈
while (!q.empty() && i < arr.size()) {
    TreeNode** nodePtr = q.front(); // 取出當前要處理的父節點
    q.pop();

    // 處理左子節點
    if (i < arr.size()) {
        if (arr[i] != EMPTY) { // 如果陣列中該位置不是空標記
            (*nodePtr)->left = new TreeNode(arr[i]); // 建立左子節點
            q.push(&((*nodePtr)->left));           // 將左子節點的位置
            推入佇列，供後續掛載它的子代
        }
        i++; // 推進陣列索引
    }

    // 處理右子節點
    if (i < arr.size()) {
        if (arr[i] != EMPTY) {
            (*nodePtr)->right = new TreeNode(arr[i]);
            q.push(&((*nodePtr)->right));
        }
        i++;
    }
}
return root;
}

void Breadth_first_search(TreeNode* root) {
    if (root == nullptr) return; // 空樹直接返回

    queue<TreeNode*> q; // 使用佇列來存放待訪問的節點
    q.push(root);

    while (!q.empty()) {
        TreeNode* current = q.front(); // 取出佇列最前面的節點
        q.pop();

        cout << current->value << " "; // 印出當前節點的值

        // 如果有左右子節點，則依序排入佇列的尾端
        if (current->left) q.push(current->left);
        if (current->right) q.push(current->right);
    }
}

int getLayerSum(int targetLayer) {
    if (root == nullptr) return 0; // 空樹的總和為 0

```

```

// 處理不合法的層數輸入 (負數)
if (targetLayer < 0) {
    cout << "Invalid layer!" << endl;
    return -1;
}

queue<TreeNode*> q;
q.push(root);
int currentLayer = 0;

// 開始進行階層走訪
while (!q.empty()) {
    // 取得當前這層共有幾個節點
    // 利用 levelSize 確保一次 for 迴圈剛好處理完一整層
    int levelSize = q.size();

    // 如果已經到達使用者想查詢的目標層
    if (currentLayer == targetLayer) {
        int sum = 0;
        // 走訪該層所有的節點，將數值累加
        for (int i = 0; i < levelSize; ++i) {
            TreeNode* current = q.front();
            q.pop();
            sum += current->value;
        }
        return sum; // 計算完畢，提早結束函式並回傳結果
    }

    // 如果還沒到達目標層，則將當前這層的所有子節點推入 Queue，準備
    // 處理下一層
    for (int i = 0; i < levelSize; ++i) {
        TreeNode* current = q.front();
        q.pop();
        if (current->left) q.push(current->left);
        if (current->right) q.push(current->right);
    }
    currentLayer++; // 處理完一層後，層數計數器 +1
}

// 如果 Queue 都空了還沒找到 targetLayer，代表輸入層數超過了樹的實際
// 高度
cout << "Layer exceeds tree height!" << endl;
return -1;
}

};

int main() {
    BinaryTree tree;

    // 定義要建立的樹狀結構陣列 (Level-order)
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, NULL, NULL, 10, 11, NULL, NULL };

    // 建立二元樹

```

```
tree.buildTree(arr);

// 1. 印出 BFS 走訪結果，驗證樹是否建立正確
cout << "BFS Result: ";
tree.Breadth_first_search(tree.root);
cout << endl;

// 2. 建立互動介面，讓使用者輸入想要查詢的層數
int target;
cout << "Please enter the layer to query, starting from 0: ";
cin >> target;

// 3. 呼叫函式計算總和
int sum = tree.getLayerSum(target);

// 4. 判斷回傳結果，若不為 -1 (代表輸入有效且未超過樹高)，則印出總和
if (sum != -1) {
    cout << "The sum of level " << target << " is: " << sum << endl;
}

// 暫停程式，方便在終端機中查看輸出結果
system("pause");
return 0;
}
```